FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES

CS054
(INTELLIGENT SYSTEMS)

EXERCISE
2

Sentiment Analysis

Group Name:	
Members:	
Section:	
Professor:	

Name	Role
John Louis M. Panganiban	Program & Tester

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: e]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design an Intelligent System based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Demonstrate understanding of Sentiment Analysis.
- Design and develop a simple program about Sentiment Analysis using Python.

IV. BACKGROUND INFORMATION

Sentiment Analysis can assist us in determining the mood and feelings of the general public as well as obtaining useful information about the setting. Sentiment Analysis is the process of assessing data and categorizing it according to the research's needs.

Sentiments can be used to gain a better understanding of different events and their consequences. It goes by various names, including "sentiment analysis," "opinion mining," "opinion extraction," "sentiment mining," "subjectivity analysis," "affect analysis," "emotion analysis," and "review mining," but they all serve the same objective. We might be able to figure out what people like or want by evaluating their feelings.



TextBlob

TextBlob is a python library and offers a simple API to access its methods and perform basic NLP tasks. Textblob's sentiment function returns polarity and subjectivity as two characteristics.

The polarity ranges from [-1,1], with 1 denoting a positive statement and -1 denoting a negative one. Subjective statements usually refer to personal feelings, emotions, or judgments, whereas objective sentences refer to facts.

Subjectivity is also a float with a value between
0 and 1.

V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY

Perform the following:

1. Add the following code:

```
from textblob import TextBlob
```

2. Declare a variable that will accept a sentence:

```
sentence = input("Ënter a sentence : ")
```

3. Next initialize TextBlob

```
s = TextBlob(sentence)
```

4. Then check the polarity of the sentence by using the code below:

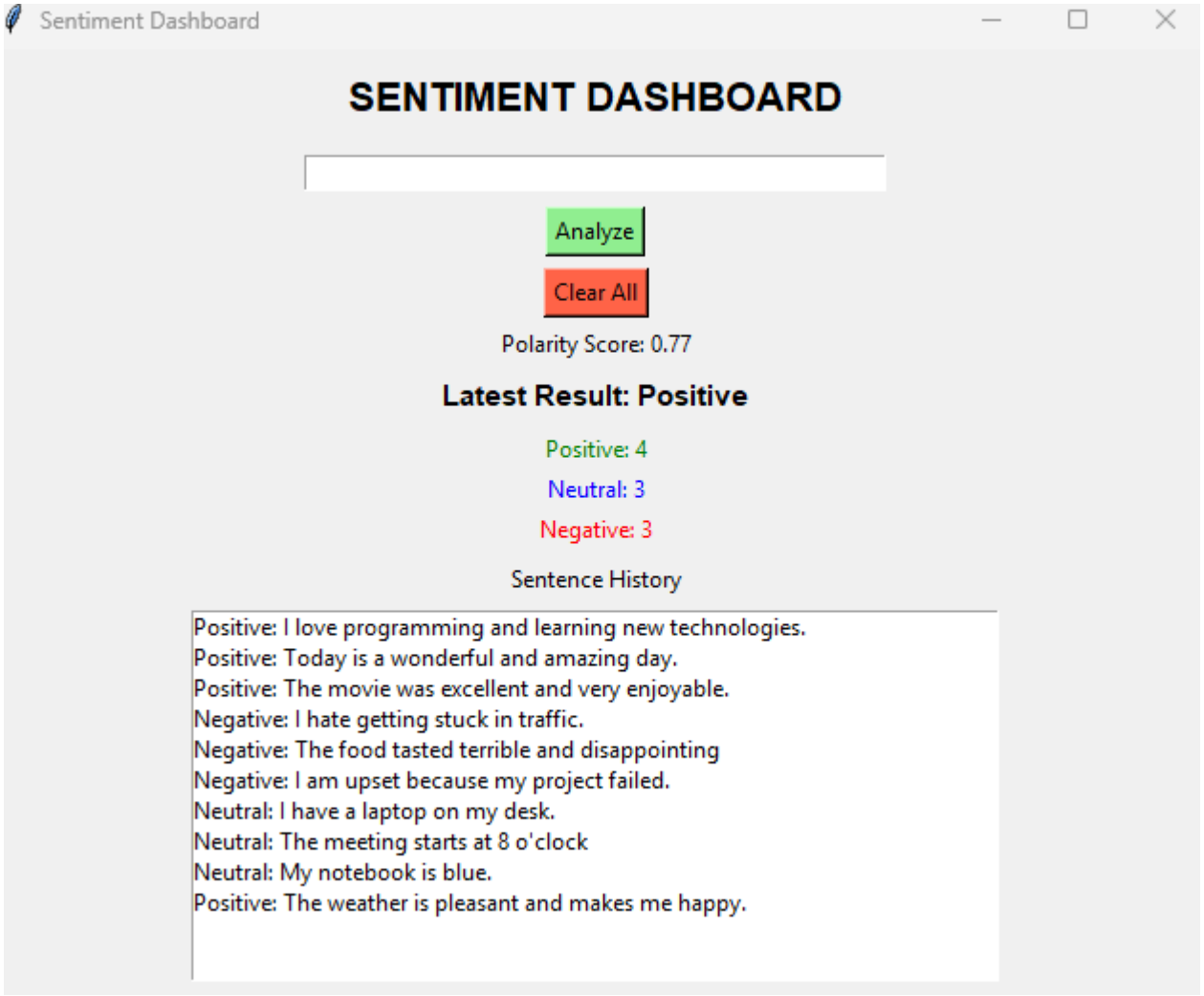
```
p = s.sentiment.polarity
```

5. Make a decision based on the polarity. If the polarity of the sentence is less than zero the sentiment is negative. If the polarity is equals to zero, then the sentiment is neutral. If the polarity is greater than zero, the sentiment is positive.

```
#Negative p<0, Neutral p=0, Positive
p>0 if p <0:
print("Negative") elif
p==0:
print("Neutral") elif p>0
and p<=1:
    print("Positive")
```

6. Make the program perform the sentiment analysis 10 times and make a summary of how many sentences are positive, neutral or negative.

OUTPUT:



CODE:

```
 from tkinter import *
from textblob import TextBlob

positive = 0
neutral = 0
negative = 0
total = 0

def analyze():
    global positive, neutral, negative, total

    if total >= 10:
        result.config(text="10 sentences reached!")
```

```
        return

    sentence = entry.get().strip()

    if sentence == "":
        return

    blob = TextBlob(sentence)
    polarity = blob.sentiment.polarity

    if polarity > 0:
        sentiment = "Positive"
        positive += 1
    elif polarity < 0:
        sentiment = "Negative"
        negative += 1
    else:
        sentiment = "Neutral"
        neutral += 1

    total += 1

    polarity_lbl.config(text=f"Polarity Score: {polarity:.2f}")
    result.config(text=f"Latest Result: {sentiment}")

    history.insert(END, f"{sentiment}: {sentence}")

    pos_lbl.config(text=f"Positive: {positive}")
    neu_lbl.config(text=f"Neutral: {neutral}")
    neg_lbl.config(text=f"Negative: {negative}")

    entry.delete(0, END)

def clear_all():
    global positive, neutral, negative, total

    positive = neutral = negative = total = 0

    history.delete(0, END)

    pos_lbl.config(text="Positive: 0")
    neu_lbl.config(text="Neutral: 0")
    neg_lbl.config(text="Negative: 0")

    polarity_lbl.config(text="Polarity Score: 0.00")
    result.config(text="Latest Result: None")

root = Tk()
root.title("Sentiment Dashboard")
root.geometry("600x500")
```

```
Label(root,text="SENTIMENT DASHBOARD",
      font=("Arial",16,"bold")).pack(pady=10)

entry = Entry(root,width=50)
entry.pack(pady=5)

Button(root,text="Analyze",
       command=analyze,bg="lightgreen").pack(pady=3)

Button(root,text="Clear All",
       command=clear_all,bg="tomato").pack(pady=3)

polarity_lbl = Label(root,text="Polarity Score: 0.00")
polarity_lbl.pack()

result = Label(root,text="Latest Result: None",
               font=("Arial",11,"bold"))
result.pack(pady=5)

pos_lbl = Label(root,text="Positive: 0",fg="green")
pos_lbl.pack()

neu_lbl = Label(root,text="Neutral: 0",fg="blue")
neu_lbl.pack()

neg_lbl = Label(root,text="Negative: 0",fg="red")
neg_lbl.pack()

Label(root,text="Sentence History").pack(pady=5)

history = Listbox(root,width=70,height=12)
history.pack()

root.mainloop()
```

CODE SCREENSHOT:

```python
from tkinter import *
from textblob import TextBlob

positive = 0
neutral = 0
negative = 0
total = 0

def analyze():
    global positive, neutral, negative, total

    if total >= 10:
        result.config(text="10 sentences reached!")
        return

    sentence = entry.get().strip()

    if sentence == "":
        return

    blob = TextBlob(sentence)
    polarity = blob.sentiment.polarity

    if polarity > 0:
        sentiment = "Positive"
        positive += 1
    elif polarity < 0:
        sentiment = "Negative"
        negative += 1
    else:
        sentiment = "Neutral"
        neutral += 1

    total += 1

    polarity_lbl.config(text=f"Polarity Score: {polarity:.2f}")
    result.config(text=f"Latest Result: {sentiment}")

    history.insert(END, f"{sentiment}: {sentence}")

    pos_lbl.config(text=f"Positive: {positive}")
    neu_lbl.config(text=f"Neutral: {neutral}")
    neg_lbl.config(text=f"Negative: {negative}")

    entry.delete(0, END)
```

```
46
47  def clear_all():
48      global positive, neutral, negative, total
49
50      positive = neutral = negative = total = 0
51
52      history.delete(0, END)
53
54      pos_lbl.config(text="Positive: 0")
55      neu_lbl.config(text="Neutral: 0")
56      neg_lbl.config(text="Negative: 0")
57
58      polarity_lbl.config(text="Polarity Score: 0.00")
59      result.config(text="Latest Result: None")
60
61  root = Tk()
62  root.title("Sentiment Dashboard")
63  root.geometry("600x500")
64
65  Label(root,text="SENTIMENT DASHBOARD",
66        font=("Arial",16,"bold")).pack(pady=10)
67
68  entry = Entry(root,width=50)
69  entry.pack(pady=5)
70
71  Button(root,text="Analyze",
72         command=analyze,bg="lightgreen").pack(pady=3)
73
74  Button(root,text="Clear All",
75         command=clear_all,bg="tomato").pack(pady=3)
76
77  polarity_lbl = Label(root,text="Polarity Score: 0.00")
78  polarity_lbl.pack()
79
80  result = Label(root,text="Latest Result: None",
81                 font=("Arial",11,"bold"))
82  result.pack(pady=5)
83
84  pos_lbl = Label(root,text="Positive: 0",fg="green")
85  pos_lbl.pack()
86
87  neu_lbl = Label(root,text="Neutral: 0",fg="blue")
88  neu_lbl.pack()
89
90  neg_lbl = Label(root,text="Negative: 0",fg="red")
```

```
91  neg_lbl.pack()
92
93  Label(root,text="Sentence History").pack(pady=5)
94
95  history = Listbox(root,width=70,height=12)
96  history.pack()
97
98  root.mainloop()
```

VII. CHALLENGE

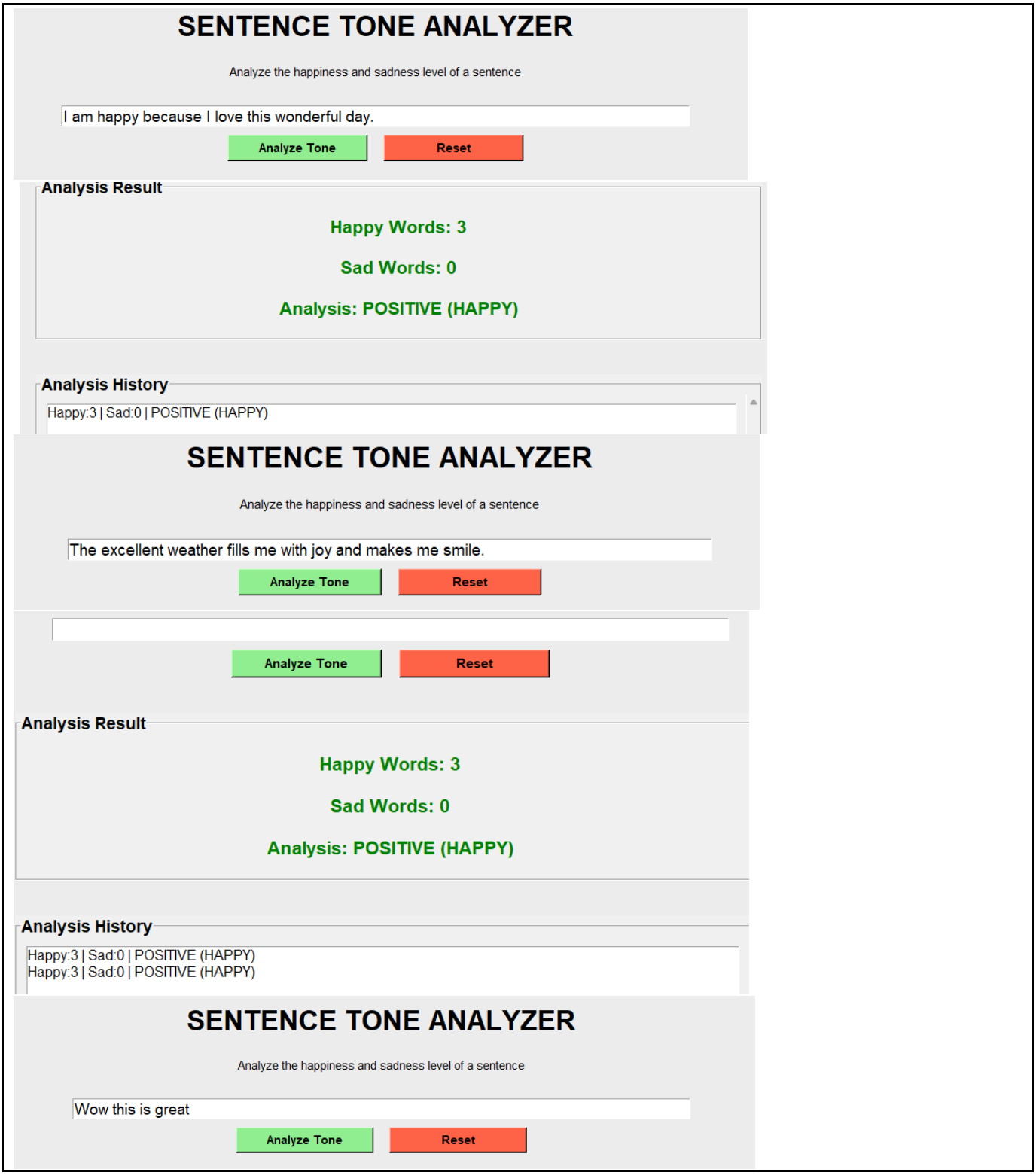
Write a program that will ask the user for a string containing a sentence. The program will then compute the happiness and sadness level of the sentence using the two functions described below. If the happiness level is higher than sadness level, then the tone of the sentence is happy. If the sadness level is higher, then the tone of the sentence is sad. Otherwise, it is neutral.

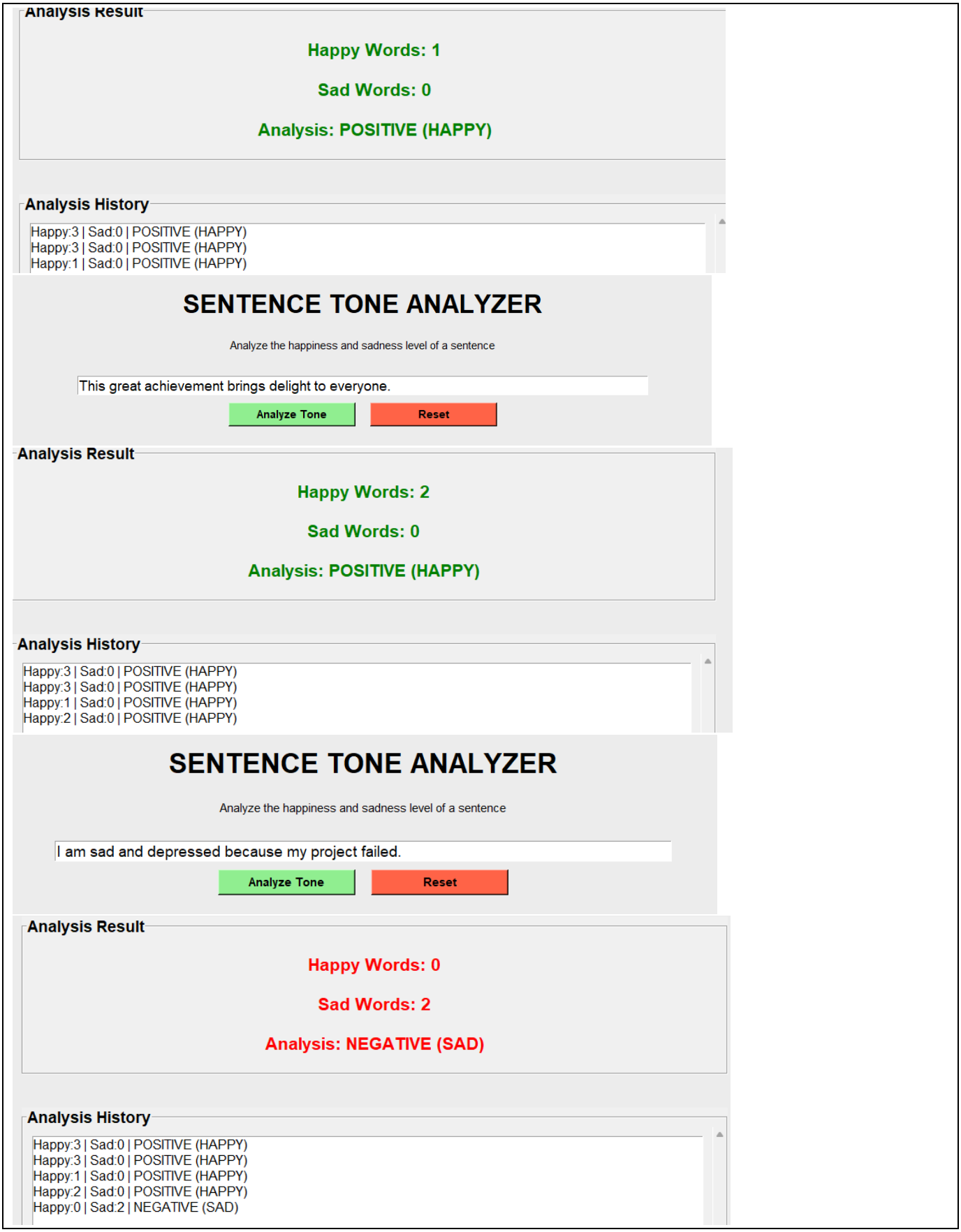
Find and print the tone of the sentence by printing the number of happy words followed by the number of sad words, followed by a simple statement of the analysis (whether positive, negative or neutral).

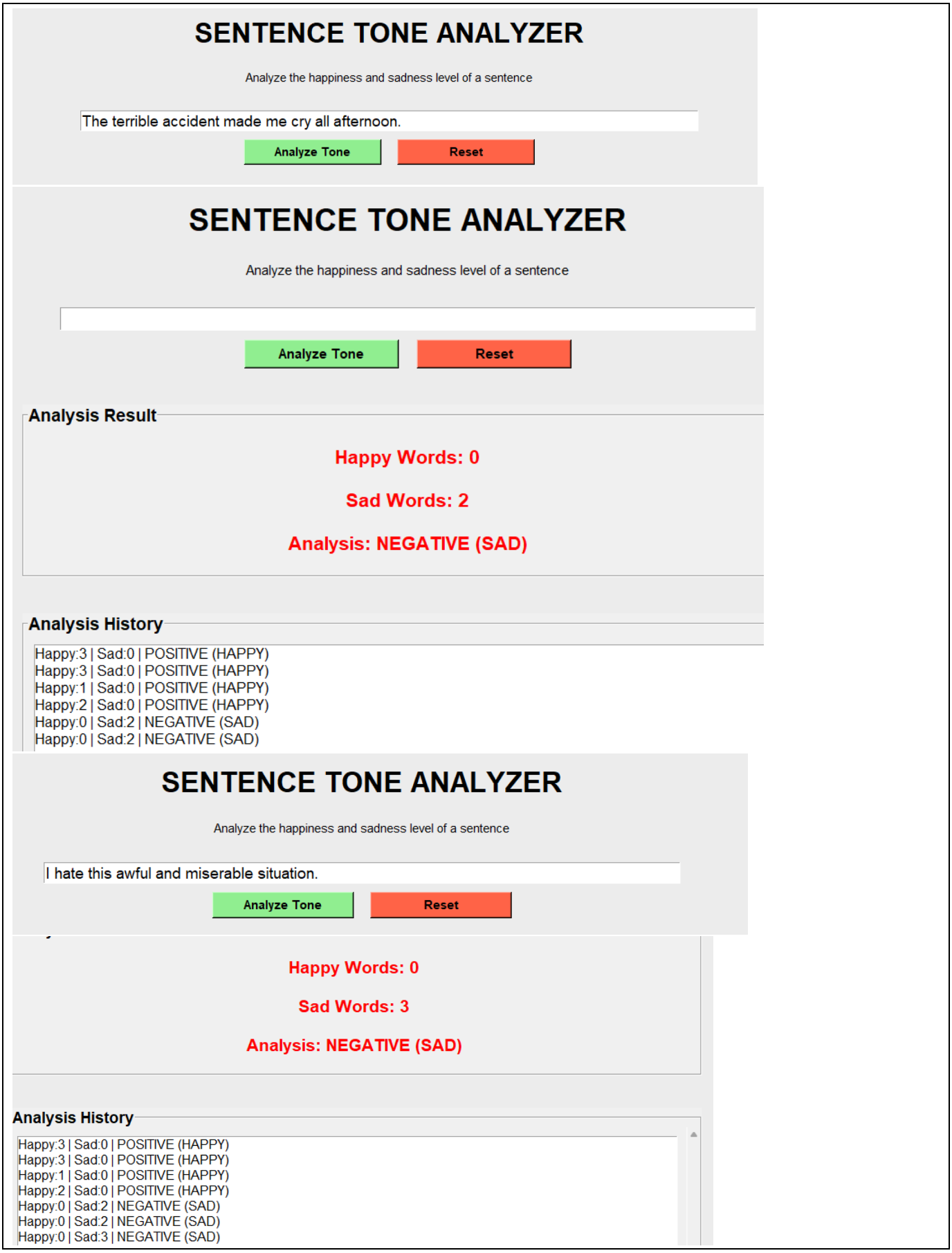
To accomplish this, you will write a function called number_happy(sentence) which returns the number of words in a given string called “sentence” that are happy. To do this, look for a public dataset containing a list of happy words. Find the total number of words in “sentence” that are in the happy words dataset.

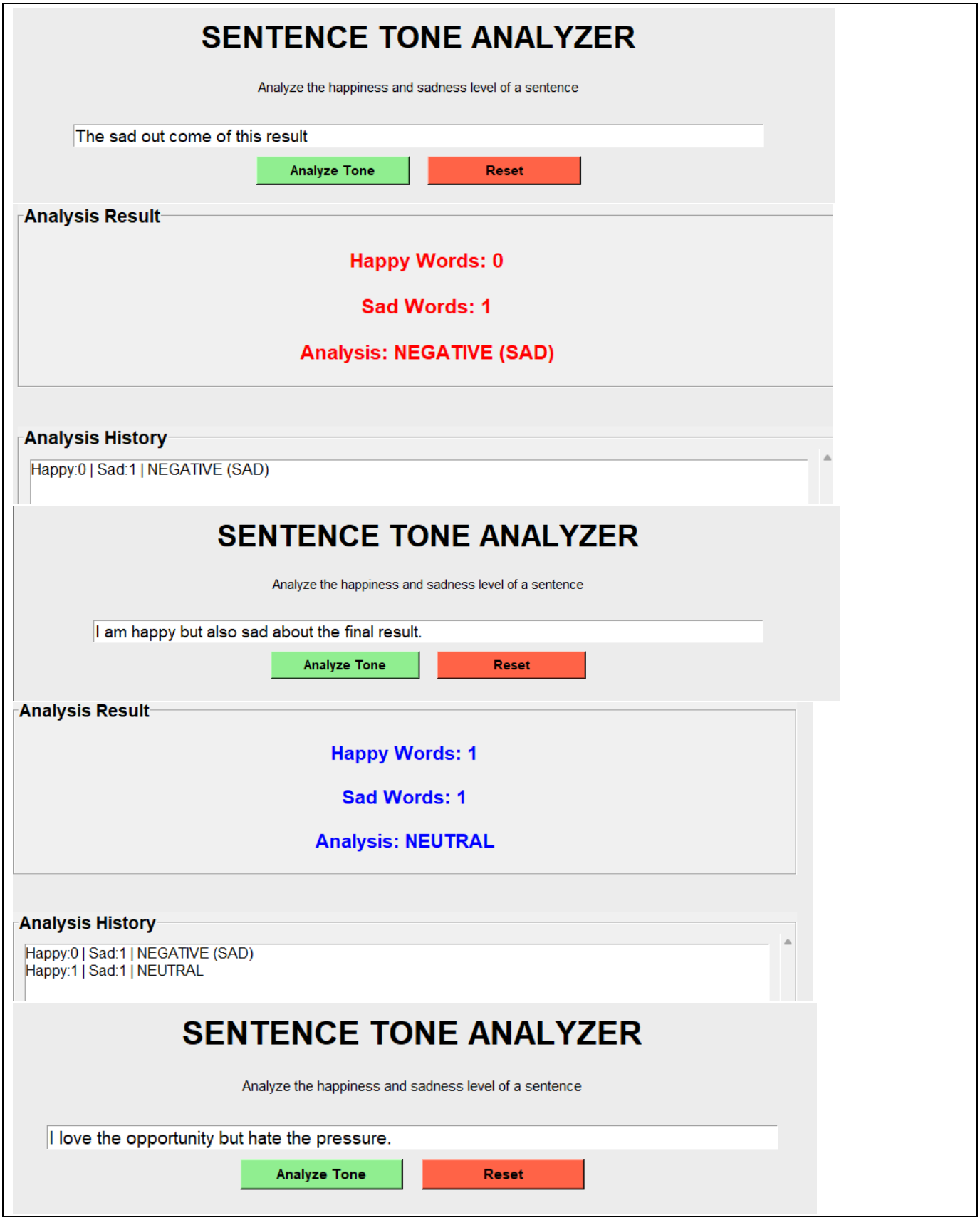
Next, write a second function called number_sad(sentence) that works the same way but instead counts the number of sad words, also from a separate dataset.

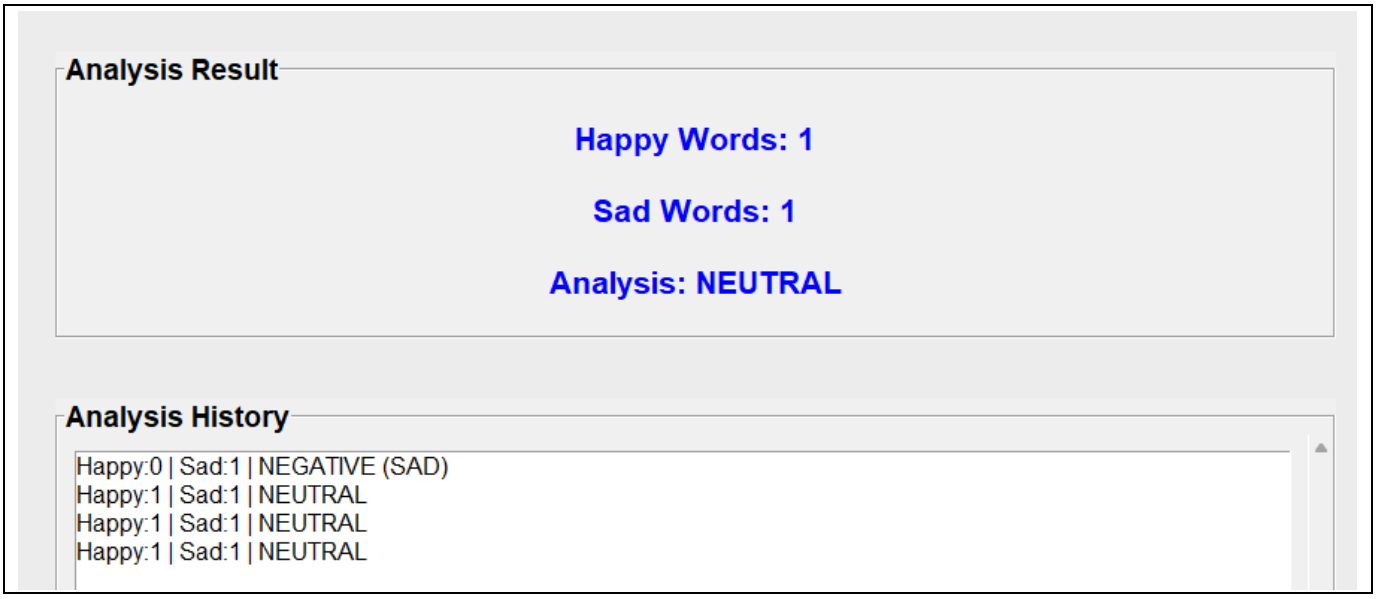
OUTPUT:


SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
I am happy because I love this wonderful day.
Analyze Tone
Reset
Analysis Result
Happy Words: 3
Sad Words: 0
Analysis: POSITIVE (HAPPY)
Analysis History
Happy:3 | Sad:0 | POSITIVE (HAPPY)
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
The excellent weather fills me with joy and makes me smile.
Analyze Tone
Reset
Analyze Tone
Reset
Analysis Result
Happy Words: 3
Sad Words: 0
Analysis: POSITIVE (HAPPY)
Analysis History
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:3 | Sad:0 | POSITIVE (HAPPY)
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
Wow this is great
Analyze Tone
Reset


Analysis Result
Happy Words: 1
Sad Words: 0
Analysis: POSITIVE (HAPPY)
Analysis History
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:1 | Sad:0 | POSITIVE (HAPPY)
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
This great achievement brings delight to everyone.
Analyze Tone
Reset
Analysis Result
Happy Words: 2
Sad Words: 0
Analysis: POSITIVE (HAPPY)
Analysis History
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:1 | Sad:0 | POSITIVE (HAPPY)
Happy:2 | Sad:0 | POSITIVE (HAPPY)
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
I am sad and depressed because my project failed.
Analyze Tone
Reset
Analysis Result
Happy Words: 0
Sad Words: 2
Analysis: NEGATIVE (SAD)
Analysis History
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:1 | Sad:0 | POSITIVE (HAPPY)
Happy:2 | Sad:0 | POSITIVE (HAPPY)
Happy:0 | Sad:2 | NEGATIVE (SAD)


SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
The terrible accident made me cry all afternoon.
Analyze Tone
Reset
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
Analyze Tone
Reset
Analysis Result
Happy Words: 0
Sad Words: 2
Analysis: NEGATIVE (SAD)
Analysis History
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:1 | Sad:0 | POSITIVE (HAPPY)
Happy:2 | Sad:0 | POSITIVE (HAPPY)
Happy:0 | Sad:2 | NEGATIVE (SAD)
Happy:0 | Sad:2 | NEGATIVE (SAD)
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
I hate this awful and miserable situation.
Analyze Tone
Reset
Happy Words: 0
Sad Words: 3
Analysis: NEGATIVE (SAD)
Analysis History
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:3 | Sad:0 | POSITIVE (HAPPY)
Happy:1 | Sad:0 | POSITIVE (HAPPY)
Happy:2 | Sad:0 | POSITIVE (HAPPY)
Happy:0 | Sad:2 | NEGATIVE (SAD)
Happy:0 | Sad:2 | NEGATIVE (SAD)
Happy:0 | Sad:3 | NEGATIVE (SAD)


SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
The sad out come of this result
Analyze Tone
Reset
Analysis Result
Happy Words: 0
Sad Words: 1
Analysis: NEGATIVE (SAD)
Analysis History
Happy:0 | Sad:1 | NEGATIVE (SAD)
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
I am happy but also sad about the final result.
Analyze Tone
Reset
Analysis Result
Happy Words: 1
Sad Words: 1
Analysis: NEUTRAL
Analysis History
Happy:0 | Sad:1 | NEGATIVE (SAD)
Happy:1 | Sad:1 | NEUTRAL
SENTENCE TONE ANALYZER
Analyze the happiness and sadness level of a sentence
I love the opportunity but hate the pressure.
Analyze Tone
Reset



CODE:

```
 import tkinter as tk

# ---------------- DATASETS ---------------- #

happy_words = [
    "happy", "joy", "cheerful", "good", "great",
    "excellent", "love", "smile", "fantastic",
    "wonderful", "delight"
]

sad_words = [
    "sad", "sorrow", "depressed", "bad", "terrible",
    "awful", "hate", "cry", "miserable",
    "tragic", "grief", "disappointed"
]

# ---------------- FUNCTIONS ---------------- #

def number_happy(sentence):
    count = 0

    for word in sentence.lower().split():
        word = word.strip(".,!?;:")
        if word in happy_words:
            count += 1

    return count

def number_sad(sentence):
    count = 0
```

```
    for word in sentence.lower().split():
        word = word.strip(".,!?;:")
        if word in sad_words:
            count += 1

    return count

def analyze_tone():

    sentence = entry.get().strip()

    if sentence == "":
        result_label.config(
            text="Please enter a sentence."
        )
        return

    happy_count = number_happy(sentence)
    sad_count = number_sad(sentence)

    if happy_count > sad_count:
        tone = "POSITIVE (HAPPY)"
        color = "green"

    elif sad_count > happy_count:
        tone = "NEGATIVE (SAD)"
        color = "red"

    else:
        tone = "NEUTRAL"
        color = "blue"

    result_label.config(
        text=f"Happy Words: {happy_count}\n\n"
             f"Sad Words: {sad_count}\n\n"
             f"Analysis: {tone}",
        fg=color
    )

    history.insert(
        tk.END,
        f"Happy:{happy_count} | Sad:{sad_count} | {tone}"
    )

    entry.delete(0, tk.END)

def clear_all():

    entry.delete(0, tk.END)
```

```
    result_label.config(
        text="Analyze a sentence to see results.",
        fg="black"
    )

    history.delete(0, tk.END)

# ---------------- WINDOW ---------------- #

root = tk.Tk()
root.title("Sentence Tone Analyzer")

# Maximized Window
root.state("zoomed")

root.configure(bg="#ECECEC")

# ---------------- HEADER ---------------- #

title = tk.Label(
    root,
    text="SENTENCE TONE ANALYZER",
    font=("Arial", 26, "bold"),
    bg="#ECECEC"
)

title.pack(pady=20)

subtitle = tk.Label(
    root,
    text="Analyze the happiness and sadness level of a sentence",
    font=("Arial", 11),
    bg="#ECECEC"
)

subtitle.pack()

# ---------------- INPUT FRAME ---------------- #

input_frame = tk.Frame(
    root,
    bg="#ECECEC"
)

input_frame.pack(pady=20)

entry = tk.Entry(
    input_frame,
    width=70,
    font=("Arial", 14)
```

```
)

entry.pack(pady=10)

button_frame = tk.Frame(
    input_frame,
    bg="#ECECEC"
)

button_frame.pack()

analyze_btn = tk.Button(
    button_frame,
    text="Analyze Tone",
    width=18,
    font=("Arial", 11, "bold"),
    bg="lightgreen",
    command=analyze_tone
)

analyze_btn.grid(row=0, column=0, padx=10)

clear_btn = tk.Button(
    button_frame,
    text="Reset",
    width=18,
    font=("Arial", 11, "bold"),
    bg="tomato",
    command=clear_all
)

clear_btn.grid(row=0, column=1, padx=10)

# ---------- RESULT FRAME ---------- #

result_frame = tk.LabelFrame(
    root,
    text="Analysis Result",
    font=("Arial", 14, "bold"),
    padx=20,
    pady=20
)

result_frame.pack(
    fill="x",
    padx=50,
    pady=20
)

result_label = tk.Label(
```

```
    result_frame,
    text="Analyze a sentence to see results.",
    font=("Arial", 16, "bold")
)

result_label.pack()

# ---------------- HISTORY FRAME ---------------- #

history_frame = tk.LabelFrame(
    root,
    text="Analysis History",
    font=("Arial", 14, "bold")
)

history_frame.pack(
    fill="both",
    expand=True,
    padx=50,
    pady=20
)

scrollbar = tk.Scrollbar(
    history_frame
)

scrollbar.pack(
    side=tk.RIGHT,
    fill=tk.Y
)

history = tk.Listbox(
    history_frame,
    font=("Arial", 12),
    yscrollcommand=scrollbar.set
)

history.pack(
    fill="both",
    expand=True,
    padx=10,
    pady=10
)

scrollbar.config(
    command=history.yview
)

root.mainloop()
```

CODE SCREENSHOT:

```python
import tkinter as tk

# ---------------- DATASETS ---------------- #

happy_words = [
    "happy", "joy", "cheerful", "good", "great",
    "excellent", "love", "smile", "fantastic",
    "wonderful", "delight"
]

sad_words = [
    "sad", "sorrow", "depressed", "bad", "terrible",
    "awful", "hate", "cry", "miserable",
    "tragic", "grief", "disappointed"
]

# ---------------- FUNCTIONS ---------------- #

def number_happy(sentence):
    count = 0

    for word in sentence.lower().split():
        word = word.strip(".,!?;:")
        if word in happy_words:
            count += 1

    return count


def number_sad(sentence):
    count = 0

    for word in sentence.lower().split():
        word = word.strip(".,!?;:")
        if word in sad_words:
            count += 1

    return count


def analyze_tone():

    sentence = entry.get().strip()

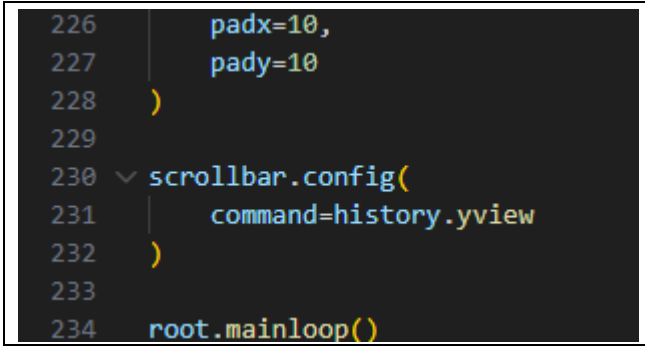
    if sentence == "":
```

```
41  def analyze_tone():
46          result_label.config(
47              text="Please enter a sentence."
48          )
49          return
50
51      happy_count = number_happy(sentence)
52      sad_count = number_sad(sentence)
53
54      if happy_count > sad_count:
55          tone = "POSITIVE (HAPPY)"
56          color = "green"
57
58      elif sad_count > happy_count:
59          tone = "NEGATIVE (SAD)"
60          color = "red"
61
62      else:
63          tone = "NEUTRAL"
64          color = "blue"
65
66      result_label.config(
67          text=f"Happy Words: {happy_count}\n\n"
68               f"Sad Words: {sad_count}\n\n"
69               f"Analysis: {tone}",
70          fg=color
71      )
72
73      history.insert(
74          tk.END,
75          f"Happy:{happy_count} | Sad:{sad_count} | {tone}"
76      )
77
78      entry.delete(0, tk.END)
79
80
81  def clear_all():
82
83      entry.delete(0, tk.END)
84
85      result_label.config(
86          text="Analyze a sentence to see results.",
87          fg="black"
88      )
```

```
92
93  # ---------------- WINDOW ---------------- #
94
95  root = tk.Tk()
96  root.title("Sentence Tone Analyzer")
97
98  # Maximized Window
99  root.state("zoomed")
100
101 root.configure(bg="#ECECEC")
102
103 # ---------------- HEADER ---------------- #
104
105 title = tk.Label(
106     root,
107     text="SENTENCE TONE ANALYZER",
108     font=("Arial", 26, "bold"),
109     bg="#ECECEC"
110 )
111
112 title.pack(pady=20)
113
114 subtitle = tk.Label(
115     root,
116     text="Analyze the happiness and sadness level of a sentence",
117     font=("Arial", 11),
118     bg="#ECECEC"
119 )
120
121 subtitle.pack()
122
123 # ---------------- INPUT FRAME ---------------- #
124
125 input_frame = tk.Frame(
126     root,
127     bg="#ECECEC"
128 )
129
130 input_frame.pack(pady=20)
131
132 entry = tk.Entry(
133     input_frame,
134     width=70,
135     font=("Arial", 14)
136 )
```

```
138    entry.pack(pady=10)
139
140    button_frame = tk.Frame(
141        input_frame,
142        bg="#ECECEC"
143    )
144
145    button_frame.pack()
146
147    analyze_btn = tk.Button(
148        button_frame,
149        text="Analyze Tone",
150        width=18,
151        font=("Arial", 11, "bold"),
152        bg="lightgreen",
153        command=analyze_tone
154    )
155
156    analyze_btn.grid(row=0, column=0, padx=10)
157
158    clear_btn = tk.Button(
159        button_frame,
160        text="Reset",
161        width=18,
162        font=("Arial", 11, "bold"),
163        bg="tomato",
164        command=clear_all
165    )
166
167    clear_btn.grid(row=0, column=1, padx=10)
168
169    # ---------------- RESULT FRAME ---------------- #
170
171    result_frame = tk.LabelFrame(
172        root,
173        text="Analysis Result",
174        font=("Arial", 14, "bold"),
175        padx=20,
176        pady=20
177    )
178
179    result_frame.pack(
180        fill="x",
181        padx=50,
```

```
182         pady=20
183     )
184
185     result_label = tk.Label(
186         result_frame,
187         text="Analyze a sentence to see results.",
188         font=("Arial", 16, "bold")
189     )
190
191     result_label.pack()
192
193     # ---------------- HISTORY FRAME ---------------- #
194
195     history_frame = tk.LabelFrame(
196         root,
197         text="Analysis History",
198         font=("Arial", 14, "bold")
199     )
200
201     history_frame.pack(
202         fill="both",
203         expand=True,
204         padx=50,
205         pady=20
206     )
207
208     scrollbar = tk.Scrollbar(
209         history_frame
210     )
211
212     scrollbar.pack(
213         side=tk.RIGHT,
214         fill=tk.Y
215     )
216
217     history = tk.Listbox(
218         history_frame,
219         font=("Arial", 12),
220         yscrollcommand=scrollbar.set
221     )
222
223     history.pack(
224         fill="both",
225         expand=True,
226         padx=10,
```



```
        padx=10,
        pady=10
    )

scrollbar.config(
        command=history.yview
    )

root.mainloop()
```

VIII. REFERENCES

- Artasanchez A. Joshi P.(2020). Artificial Intelligence with Python: You Complete Guide to Building Intelligent apps using Python 3.x 2nd Edition. Packt Publishing
- Attaway, Stormy (2013). MATLAB ®. A Practical Introduction to Programming and Problem Solving Third Edition. Elsevier, Inc.

Online Sources:

- https://www.cuelogic.com/blog/role-of-python-in-artificial-intelligencehttps://monkeylearn.com/sentimentanalysis
- https://towardsdatascience.com/a-beginners-guide-to-sentiment-analysis-in-python-95e354ea84f6
- https://research.google.com/colaboratory
- https://www.youtube.com/watch?v=RLYoEyIHL6A